

Práctica 3: Unidad Aritmético Lógica

Objetivos generales

- El alumno practicará el proceso de diseño de circuitos combinacionales

Objetivos particulares

- El diseño modular de los circuitos combinacionales.
- Los componentes básicos de una unidad aritmético lógica, así como el diseño de la misma.

Desarrollo

Sumador

Para crear un sumador y restador de 8 bits aprovecharemos la capacidad del diseño modular de los circuitos combinacionales y comenzaremos diseñando un sumador de 1 bit, posteriormente integraremos 8 sumadores de 1 bit y finalmente agregaremos otros circuitos para obtener el restador.

Sumador de 1 bit

Nuestro sumador de 1 bit será un sumador completo o full adder, este contará con las entradas A, B y C, para los operandos y el acarreo de entrada respectivamente y las salidas O y C_{i+1} para el resultado de la operación y el acarreo de salida.

Sumador de 8 bits

Obtenemos el sumador de 8 bits conectando 8 sumadores de 1 bit de la siguiente forma: la salida C_{i+1} del sumador n se conecta a la entrada C_{i+1} del sumador $n+1$. Así cada A_n , B_n y O_n corresponderán al n -ésimo bit del operando A, el operando B y la salida Resultado, comenzando con el bit menos significativo. Queda sin conexión la entrada C_i del primer sumador, la del bit menos significativo y la salida C_{i+1} del último, la del bit más significativo; la primera nos será de utilidad más adelante en el desarrollo de la sustracción y la segunda nos sirve para detectar el desbordamiento aritmético, la cual solo se deberá activar cuando se realiza la adición.

Sustracción

Primero adoptaremos la siguiente convención: Para representar los inversos aditivos en nuestro sumador, usaremos la representación de complemento a dos. Entonces en el diseño del circuito que realizará la sustracción, podemos aprovechar los circuitos que ya tenemos, esto es para obtener $A - B$ sumaremos al minuendo A el inverso aditivo del sustraendo B. Gracias a la convención adoptada, para obtener el inverso aditivo del número, debemos invertir cada bit de la entrada B y sumarle 1, por lo que para obtener la sustracción, la ALU debe llevar a cabo la operación $A + \bar{B} + 1$ en donde $\bar{B}$ es la entrada B con todos sus bits negados.

Ejercicios:

1. Desarrollar un sumador completo de 1 bit

2. Desarrollar un sumador completo de 8 bits
3. Desarrollar un sumador y un restador de 8 bits
4. Usando un multiplexor, desarrollar una ALU de 4 bits, que pueda elegir entre 8 operaciones:
 - a. AND: Operación lógica AND
 - b. OR: Operación lógica OR
 - c. ADD with carry: Suma con acarreo de entrada.
 - d. SUB with borrow: Resta con acarreo.
 - e. NEGATE: A se sustrae de 0, cambia su símbolo (convierte positivo en negativo y negativo en positivo)
 - f. Increment: Suma 1 a A
 - g. Decrement: Resta 1 a A
 - h. PASS: Pasa los bits sin alterar.
5. Añade banderas a el Multiplexor del ejercicio 4:
 - a. Zero flag: Se activa si el resultado de la Alu es 0
 - b. Overflow Flag: Se activa si el resultado supera el número de bits del resultado.
 - c. Negate Flag: Se activa si el resultado de la ALU es negativo (Solo se activa en restas)
 - d. Logic/Arithmetic Flag: Cambia de 0 a 1 en caso de estar realizando una operación Aritmética o Lógica.

Preguntas:

1. En el contexto de las ALU's, explica que son los SIMD, MIMD, La Execution Units (EU's) y los Address Generation Units (AGU's)

El SIMD es una arquitectura que permite aplicar una sola instrucción a variedad de datos , es decir la operación se ejecuta a muchos datos a la vez, eso aprovecha el paralelismo para realizar los cálculos.

Esto es especialmente útil para procesamiento de imágenes, aquí el procesador envía una sola instrucción a las ALU's y ellas trabajan en diferentes datos al mismo tiempo, trabajando en paralelo.

En el MIMD cada procesador ejecuta instrucciones diferentes y trabaja sobre distintos datos lo que permite hacer multitareas, por ejemplo en esta arquitectura es posible que cada núcleo tenga sus propias ALU's para ejecutar instrucciones de forma paralela, la diferencia con el SIMD es que en lugar de aplicar una instrucción a varios datos aquí muchas instrucciones se ejecutan en distintos datos.

Las Execution Units (EU's) como su nombre lo dice es donde se ejecutan las operaciones dadas por las instrucciones, aquí es donde se encuentran las ALU's de hecho, así como otras unidades como las de punto flotante, las vectoriales y las de almacenamiento. Las Address Generation Units (AGU's) son aquellas que calculan las direcciones de memoria para que nos sea posible acceder a los datos , para realizar sus cálculos se apoyan de los registros, offsets e índices.

Entonces cuando una Alu quiere realizar una operación el procesador puede determinar dónde están los datos antes de que se realice la operación.

2. ¿Qué es el Carry-Look-Ahead Adder? Muestra su diagrama.

Es un sumador que se usa para reducir el tiempo en el que se propaga el acarreo, pues al sumar los bits se debe de pasar al siguiente más significativo, y como cada sumador espera al anterior siempre existe un retraso, así lo que hace el Carry-Look-Ahead Adder es que calcula desde antes los acarreos dadas las entradas asu no tiene que esperar a que se pasen de una a otra.

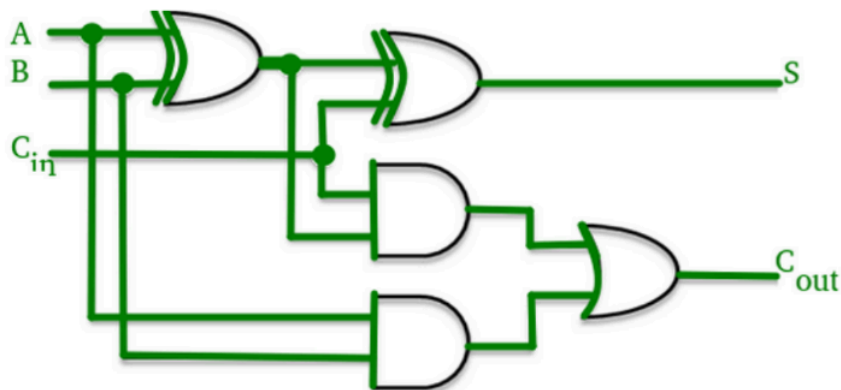
Por ejemplo si los dos bits son 1 siempre habrá un acarreo a eso se le dice Generate:

$$G_i = A_i B_i$$

Y en el caso donde uno de los bits es 1 , en el dado caso de tener un acarreo del bit anterior lo pasa directamente al siguiente:

$$P_i = A_i \oplus B_i$$

Eso es el propagate.



A	B	C	C +1	Condition
0	0	0	0	No Carry Generate
0	0	1	0	
0	1	0	0	
0	1	1	1	No Carry Propagate
1	0	0	0	
1	0	1	1	
1	1	0	1	Carry Generate
1	1	1	1	

Fuente: GeeksforGeeks, 2024

3. Las Alu modernas suelen estar conectadas en **Pipelines** ¿Qué significa conectar una ALU en **Pipelines**? ¿Cuál es la ventaja y cuál es el riesgo o desafío potencial de hacerlo?

Lo que implica conectar una Alu a una pipeline es que se va a dividir las operaciones por etapas, esto lo que permite es que las instrucciones se vayan procesando en diferentes etapas en el hardware.

Así por ejemplo si se ejecuta una instrucción en la ejecución en la ALU otra puede estar siendo obtenida de memoria, así al mismo tiempo se pueden ir procesando varias instrucciones.

Su principal ventaja es que aumenta razonablemente el rendimiento del procesador, pues al ejecutar varias instrucciones al mismo tiempo pero en distintas etapas eso le da espacio al procesador para completar esas instrucciones en menos tiempo, además de que disminuye el trabajo que necesita hacer.

Ahora, la principal desventaja es que como las operaciones pueden depender de instrucciones anteriores puede provocar errores en lo que hay conflictos con los datos, para esto se usa stalling que es detener por un determinado tiempo la pipeline.

Referencias:

GeeksforGeeks. (2026, 14 enero). Carry LookAhead Adder. GeeksforGeeks.

<https://www.geeksforgeeks.org/digital-logic/carry-look-ahead-adder/>

www.naukri.com (s. f.). Code 360 by Coding Ninjas. 2024 Naukri.com.

<https://www.naukri.com/code360/library/pipelining-in-computer-architecture>

GeeksforGeeks (2024, 12 septiembre). Difference between SIMD and MIMD.

GeeksforGeeks.

<https://www.geeksforgeeks.org/computer-organization-architecture/difference-between-simd-and-mimd/>

Client challenge. (s. f.).

<https://es.scribd.com/document/842646596/Pipelining-in-Computer-Architecture>